Trabajo práctico 2: revisión del proyecto

Lógica, pasos y decisiones

Alejandro Quesada Jose Herrera Manfred Azofeifa

19 de octubre de 2026

Resumen

Este documento explica qué se ha hecho en el Trabajo práctico 2, en qué orden, con qué lógica y por qué se tomó cada decisión. Es una guía interna para que cualquier integrante del equipo (o un agente de IA) entienda el estado del proyecto sin leer todo el código. No reemplaza al informe que se entrega (`main.tex`), que responde el enunciado. Se actualiza cada vez que se completa un apartado o se toma una decisión.

Índice

1. El problema **2**

2. Organización del repositorio **2**
2.1. Estructura . . . 2
2.2. Decisión: un archivo por punto . . . 2
2.3. Regla de independencia y el notebook como *main* . . . 3
2.4. Contratos entre archivos . . . 3
2.5. Figuras, tablas y pruebas . . . 3

3. Protocolo experimental común **3**

4. Punto 1: regresión logística **4**
4.1. 1.1 Evaluación del modelo (`forward`) . . . 4
4.2. 1.2 Entrenamiento (`train_logistic_regression`) . . . 4
4.3. Pruebas unitarias de 1.1 y 1.2 . . . 4
4.4. 1.3 Verificación con datos sintéticos . . . 4
4.5. 1.4 Características del codificador y 30 corridas . . . 5
4.6. 1.5 PCA (pendiente) . . . 7

5. Punto 2: detección de anomalías (pendiente) **7**

6. Punto 3: modelos de lenguaje (pendiente) **7**

7. Registro de decisiones **8**

8. Verificación y cómo reproducir **8**

1. El problema

El enunciado pide detectar mensajes SMS no deseados (*spam*) del dataset público SMSSpamCollection del repositorio UCI, con tres enfoques:

1. **Punto 1 (60 puntos):** regresión logística propia en PyTorch: evaluación (`forward`, 1.1), entrenamiento (1.2), verificación con datos sintéticos (1.3), clasificación de los mensajes a partir de las características de un codificador (1.4) y reducción de dimensionalidad con PCA (1.5).

2. **Punto 2 (30 puntos):** detección de anomalías con densidades por histograma y un umbral de verosimilitud.

3. **Punto 3 (10 puntos y 20 extra):** clasificación con modelos grandes de lenguaje y, como extra, ajuste fino con LoRA o DoRA.

La entrega es el martes 20 de octubre de 2026: un `.zip` con el código fuente LaTeX, el PDF y el notebook documentado.

2. Organización del repositorio

2.1. Estructura

Ruta	Contenido
`src/tp2/punto1.py`	Punto 1 completo (1.1 a 1.5).
`src/tp2/punto2.py`	Punto 2 (2.1 y 2.2).
`src/tp2/punto3.py`	Punto 3 (3.1 a 3.3).
`src/tp2/utils.py`	Infraestructura compartida: datos, particiones, métricas, corridas, figuras, tablas y pruebas en el notebook.
`notebooks/tp2_detector_spam.ipynb`	El *main*: único lugar donde se combinan los archivos.
`tests/`	Un archivo de pruebas por punto, más `test_utils.py` y la prueba de independencia.
`docs/report/`	Informe (`main.tex`, una sección por punto) y este documento.
`results/`	Figuras (PNG) y tablas (CSV) que exporta el notebook.
`TODO.md, AGENTS.md`	Guía de trabajo para personas y para agentes de IA.
`docs/report/review.tex, review.pdf`	Este documento y su PDF compilado.

2.2. Decisión: un archivo por punto

El proyecto partió del esqueleto del TP1 (un notebook con un `utils.py`), pero con la implementación fuera del notebook. Primero se dividió en un módulo por apartado (once archivos); luego el equipo decidió dejar **un archivo por punto** (`punto1.py`, `punto2.py`, `punto3.py`) más `utils.py`. Con tres personas, la asignación es directa: cada quien es dueño de su archivo, sus pruebas, su sección del notebook y su sección del informe, y los conflictos de Git quedan limitados a la celda del notebook que integra su parte. Se eligieron archivos `.py` y no un notebook por punto porque Git los fusiona sin problemas y Doxygen los documenta directamente.

2.3. Regla de independencia y el notebook como *main*

Ningún archivo de `src/tp2/` importa a otro del paquete, ni siquiera a `utils.py`. Cuando un punto necesita algo de otro, el notebook se lo entrega como argumento: por ejemplo, las gráficas de 1.3 reciben una *función* de probabilidad y no el modelo, y la calibración de 1.4 recibe la métrica `utils.f1_score` como argumento. Así cada archivo se programa y se prueba por separado. La prueba `test_module_independence.py` analiza los `import` de cada archivo y falla si alguno rompe la regla; también falla si una dependencia opcional (`sentence-transformers`, `transformers`, `peft`) se importa al inicio de un archivo en lugar de dentro de la función que la usa.

2.4. Contratos entre archivos

Dato	Formato
Características	tensor `float32` de forma (n, D), en el orden del dataset.
Etiquetas y predicciones	tensor `int64` de forma $(n,)$: *ham* $= 0$, *spam* $= 1$.
Clase positiva	spam: es la clase del F1-score y la anomalía del punto 2.
Una corrida	función `run(seed) ->dict` de métricas.
Figuras	cada función devuelve la figura sin mostrarla; el notebook la muestra y la exporta con `utils.show_figure`.

2.5. Figuras, tablas y pruebas

- **Exportación.** Cada figura mostrada con `utils.show_figure` se guarda en `results/figures/` con un nombre derivado de su título, y cada tabla mostrada con `utils.display_table` se guarda en `results/tables/`. Al ejecutar la celda de configuración se borran los archivos anteriores, así que los resultados siempre corresponden a la última ejecución.
- **Estilo de las figuras.** Azul para la clase 0 (ham) y naranja para la clase 1 (spam), dos colores que también se distinguen con daltonismo; cada clase lleva además su propio marcador (círculo o triángulo). Ejes y cuadrícula discretos, texto en tinta neutra y un subtítulo que explica cómo leer cada figura.
- **Pruebas.** Se ejecutan con `pytest` y también dentro del notebook con `utils.run_tests`. Una prueba o función sin terminar lanza `NotImplementedError` y se reporta como *pendiente*, no como fallida, para que el avance sea visible.

3. Protocolo experimental común

- **Métricas.** Con TP, FP, FN y TN los verdaderos y falsos positivos y negativos (positivo = spam):

$$\text{tasa de aciertos} = \frac{TP + TN}{n}, \qquad F_1 = \frac{2\,TP}{2\,TP + FP + FN}.$$

  Como solo el 13 % de los mensajes es spam, un clasificador que diga siempre "ham" tendría una tasa de aciertos cercana a 0.87 y un F_1 de 0; por eso el F_1 es la métrica que más informa.
- **Particiones.** `utils.stratified_split` baraja los índices de cada clase por separado y toma el 70 % de cada una para entrenamiento, de modo que las dos particiones conservan la proporción de spam.

- **Semillas.** La corrida k usa la semilla `SEED` $+k-1$ (`SEED` $= 2026$) para la partición y para la inicialización del modelo, así que todo el experimento es reproducible.
- **Repeticiones.** Los puntos 1.4, 1.5 y 2.2 repiten el experimento en 30 particiones distintas; el punto 3.1 usa 30 muestras estratificadas del 30 % del dataset.

4. Punto 1: regresión logística

4.1. 1.1 Evaluación del modelo (`forward`)

El modelo es $p(y = 1 \mid x) = \sigma(x^\top w + b)$. `forward` calcula los logits $z = Xw + b$ de todas las muestras con un producto matriz-vector y les aplica una sigmoide propia. La forma directa $1/(1 + e^{-z})$ calcula e^{1000} cuando $z = -1000$ y su gradiente con Autograd sale indefinido (NaN). Por eso la sigmoide se evalúa con $e^{-|z|} \in (0, 1]$: $1/(1 + e^{-|z|})$ si $z \geq 0$ y $e^{-|z|}/(1 + e^{-|z|})$ si $z < 0$, que son la misma función sin infinitos.

4.2. 1.2 Entrenamiento (`train_logistic_regression`)

- **Pérdida:** entropía cruzada binaria media.
- **Optimización:** descenso del gradiente por lotes completos, implementado a mano (sin `torch.nn` ni `torch.optim`), con el gradiente exacto $\nabla_w L = X^\top(p - y)/n$ y $\partial L/\partial b = \overline{p - y}$, donde p se evalúa con `forward`. Se prefirió la fórmula cerrada a Autograd porque es exacta, transparente y no depende de que el grafo de cómputo sea estable.
- **Inicialización:** pesos normales con desviación 0.01 generados con la semilla y sesgo en cero. La misma semilla da siempre el mismo modelo.
- **Historial:** la pérdida inicial y la de cada época, calculada como $\log(1 + e^z) - yz$ para que no se indefina si p vale 0 o 1.
- **Costo:** con datos del tamaño de 1.4 ($3\,900 \times 384$), 1000 épocas tardan unos 0.7 segundos en CPU.

4.3. Pruebas unitarias de 1.1 y 1.2

Hay dos pruebas por función (`tests/test_punto1.py`): pesos nulos dan probabilidad 0.5; la salida coincide con la sigmoide calculada a mano y es estable con logits de ± 1000; el primer paso de entrenamiento coincide con el gradiente de referencia de Autograd y la pérdida baja en cada época; y el modelo separa sin errores datos separables, de forma reproducible. Para comprobar que las pruebas sirven, se introdujeron a propósito errores en la implementación (sigmoide ingenua, signo invertido, sin sesgo, gradiente sin promediar): cada error hizo fallar al menos una prueba.

4.4. 1.3 Verificación con datos sintéticos

Lógica. Antes de usar los mensajes se verifica el regresor en un caso con respuesta conocida. La frontera verdadera es fija, $0.6\,x_1 + 0.8\,x_2 - 0.5 = 0$; como $(0.6, 0.8)$ es un vector unitario, $w^{*\top}x + b^*$ es directamente la distancia con signo a la recta. Cada punto se coloca a una distancia $|d| = 1 + 0.5\,|z|$ (con z normal estándar) del lado de su clase, y a una posición normal a lo largo de la recta. Queda una franja vacía de ancho 2 entre las clases, de modo que los datos son separables por construcción.

Decisión. Se fijó la frontera (en lugar de sortearla) para poder comparar la frontera aprendida con la verdadera en tablas y figuras.

Pasos. 200 puntos de entrenamiento (semilla 2026) y 200 de prueba (semilla 2027); entrenamiento con tasa 0.1 y 1000 épocas; para mostrar el proceso se reentrena con 0, 10, 50 y 200 épocas, lo que reproduce exactamente esos estados del entrenamiento porque es determinista.

Resultados. El modelo recupera la frontera verdadera: tras 10 épocas clasifica bien todo el entrenamiento y el ángulo entre w y w^* baja de 26° (época 0) a 0.73°, y tras 1000 épocas a 0.01°; la pérdida baja de 0.681 a 0.009 sin oscilar. En los 200 puntos de prueba la tasa de aciertos y el F_1 valen 1 (Figura 1). Dos detalles enseñan cómo aprende la regresión logística con datos separables: $\|w\|$ crece sin parar (de 0.02 a 3.84), porque la pérdida se puede seguir reduciendo haciendo más abrupta la transición de probabilidad; y el sesgo normalizado (-0.35 frente a -0.5) avanza mucho más lento que la orientación, porque cualquier recta dentro de la franja vacía separa perfectamente y el descenso del gradiente solo se acerca muy despacio a la de margen máximo.

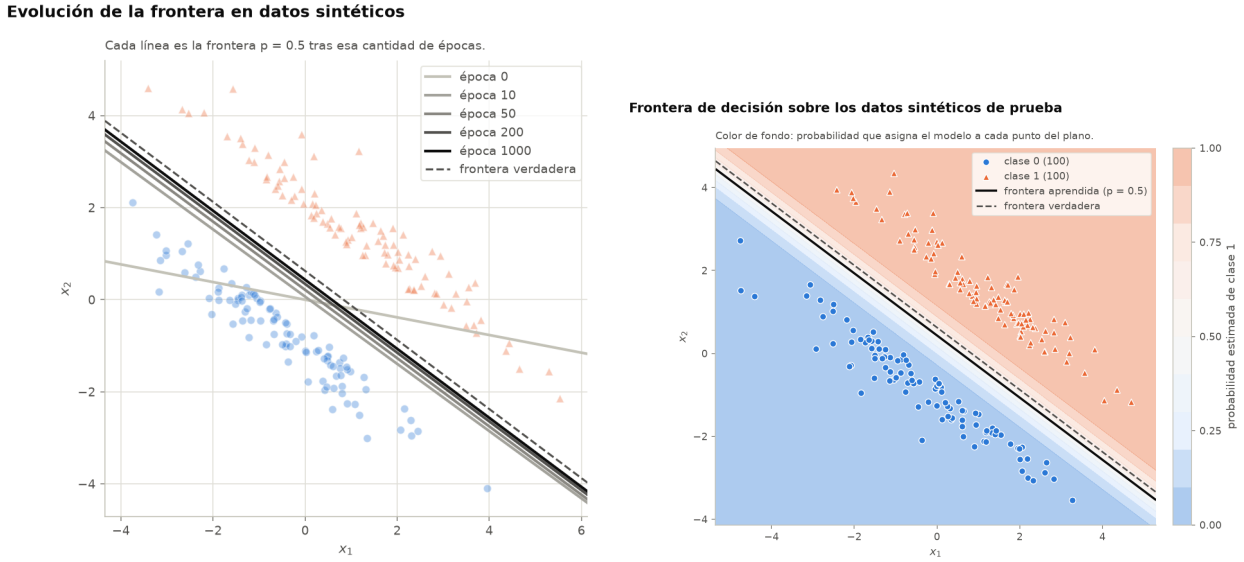


Figura 1: Izquierda: frontera tras 0, 10, 50, 200 y 1000 épocas. Derecha: probabilidad del modelo final sobre los datos de prueba.

4.5. 1.4 Características del codificador y 30 corridas

Codificador. El equipo eligió `sentence-transformers/all-MiniLM-L6-v2` (19 de octubre de 2026) frente a BERT base y DistilBERT: produce vectores de 384 dimensiones de norma 1 pensados para representar oraciones completas, ocupa unos 90 MB y codifica los 5 574 mensajes en CPU en poco tiempo. Las características se calculan una vez y se guardan en `data/processed/`, porque no dependen de la partición.

Calibración. Con vectores de norma 1 cada coordenada vale del orden de 0.05, así que el gradiente es pequeño y la tasa 0.1 de 1.3 aprende muy despacio. El equipo decidió calibrar la tasa de aprendizaje y las épocas con una validación: del 70 % de entrenamiento de la primera partición se separa un 20 % estratificado, se entrena cada combinación de la cuadrícula (tasas 0.5 a 50, de 250 a 4000

épocas) con el 80 % restante y se elige la de mayor F_1 en la validación. El F_1 de una validación con 105 spam cambia a saltos y muchas combinaciones empatan, así que el empate se resuelve con la menor entropía cruzada en la validación. Resultado: tasa 10 y 2000 épocas ($F_1 = 0.961$). Se observa que lo que cuenta es el producto tasa × épocas (10×2000, 5×4000 y 20×1000 dan lo mismo), que con poco entrenamiento el F_1 cae hasta 0.42 y que con demasiado (tasa 50 y 4000 épocas) la pérdida de validación sube y el F_1 baja a 0.951: un sobreajuste leve. Con la tasa 0.1 sin calibrar, una ejecución preliminar dio un F_1 medio de solo 0.23 (Figura 2).

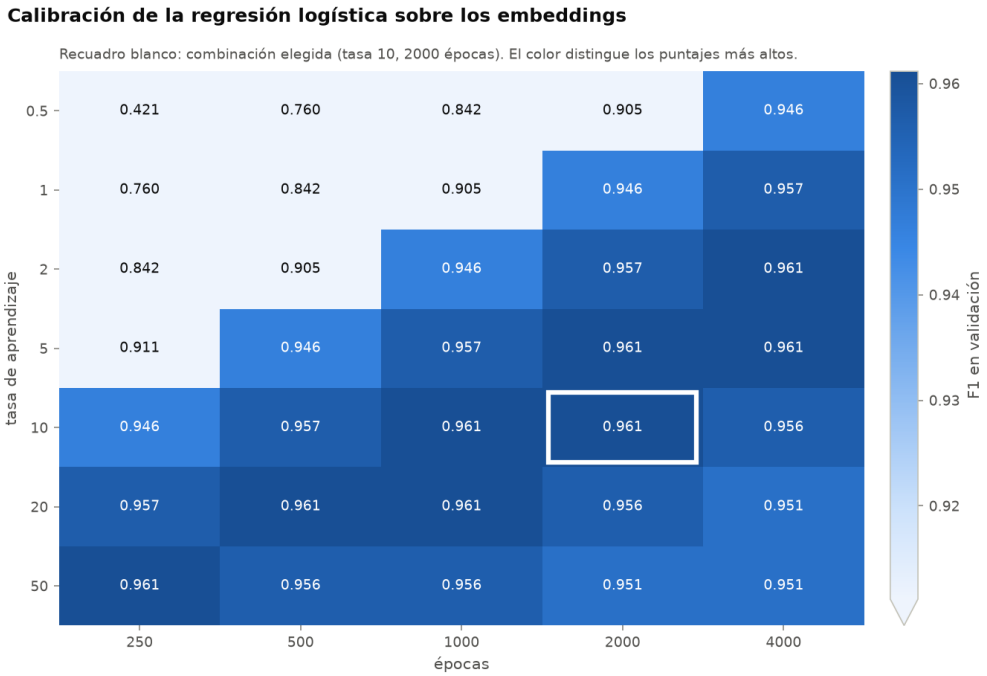


Figura 2: F_1 en validación de cada combinación; el recuadro blanco es la elegida.

Resultados de las 30 corridas. Tasa de aciertos 0.988 ± 0.002 (entre 0.984 y 0.992) y F_1 0.953 ± 0.008 (entre 0.939 y 0.968): la dispersión es pequeña, así que el resultado no depende de la partición. En promedio, de 1 448 ham de prueba solo 6.2 se marcan como spam (0.4 %) y de 224 spam se escapan 14.6 (6.5 %): precisión 0.971 y exhaustividad 0.935 (Figura 3). Los errores son sobre todo spam que pasa, el lado preferible para un filtro. Un clasificador que dijera siempre "ham" tendría 0.866 de aciertos y $F_1 = 0$.

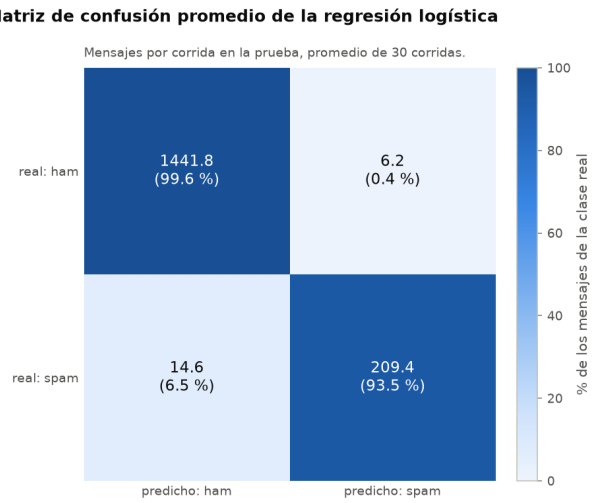


Figura 3: Matriz de confusión promedio de las 30 corridas.

Limitación. La calibración usa datos de la primera partición, que en otras corridas pueden caer en la prueba. Solo se eligieron dos números con ellos, así que el sesgo optimista es pequeño, pero existe.

4.6. 1.5 PCA (pendiente)

Diseño previsto: en cada corrida el PCA se ajusta solo con el entrenamiento (para no filtrar información de la prueba) y se reduce a 50 % ($k = 192$), 25 % ($k = 96$) y 2 dimensiones. La dimensión con mejor resultado será la D óptima que usa el punto 2.

5. Punto 2: detección de anomalías (pendiente)

Diseño previsto: cada dimensión del PCA óptimo se modela con un histograma p_d; el puntaje de un mensaje es $\sum_d \log p_d(x_d)$ y un puntaje menor que el umbral marca spam. De forma provisional, el notebook construye las densidades solo con los mensajes ham del entrenamiento, que representan la distribución "normal"; esta decisión y el criterio del umbral siguen pendientes del equipo.

6. Punto 3: modelos de lenguaje (pendiente)

Diseño previsto: con un modelo fijo y decodificación determinista, la predicción de un mensaje no depende de la muestra, así que cada mensaje se clasifica una sola vez por modelo y las 30 muestras del 30 % reutilizan esas predicciones. El resultado es el mismo que clasificar cada muestra, con 30 veces menos inferencias. Faltan por decidir los modelos, el prompt y el tratamiento de respuestas ambiguas.

7. Registro de decisiones

Fecha	Decisión	Motivo
17/10/2026	Implementación en archivos `.py` y notebook como *main*.	Git fusiona `.py` sin problemas; varias personas trabajan en paralelo.
17/10/2026	Un archivo por punto más `utils.py`.	Asignación directa entre tres personas (decisión del equipo).
17/10/2026	Ningún archivo importa a otro.	Cada punto se programa y prueba por separado; una prueba lo verifica.
17/10/2026	`TODO.md` para personas y `AGENTS.md` para agentes, siempre actualizados juntos.	Pedido del equipo.
17/10/2026	Sigmoide estable y descenso del gradiente con gradiente exacto (1.1 y 1.2).	Implementación propia, sin NaN con logits grandes.
19/10/2026	Frontera verdadera fija en 1.3.	Permite comparar la frontera aprendida con la verdadera.
19/10/2026	Codificador `all-MiniLM-L6-v2` (1.4).	Decisión del equipo: buena representación de oraciones y rápido en CPU.
19/10/2026	Calibración por F_1 en una validación del entrenamiento (1.4).	Decisión del equipo: la tasa 0.1 no sirve con vectores de norma 1.
19/10/2026	Empates de la calibración resueltos con la entropía cruzada en validación (1.4).	Con 105 spam el F_1 empata en muchas combinaciones; el primer criterio (menos épocas) elegía el borde de la cuadrícula (tasa 50).
19/10/2026	Tasa 10 y 2000 épocas (1.4).	Resultado de la calibración.
19/10/2026	Cada corrida guarda su matriz de confusión (1.4).	Muestra qué tipo de error comete el modelo, no solo cuántos.
19/10/2026	Este documento se versiona como `.tex` y como PDF, y se actualiza en cada cambio junto con `TODO.md` y `AGENTS.md`.	Pedido del equipo: tener siempre a mano una versión legible y al día.
19/10/2026	LaTeX con MiKTeX en Windows y `./build_pdf.sh review` para compilar este documento.	No había LaTeX instalado; MiKTeX instala los paquetes que faltan al compilar.
19/10/2026	`build_pdf.sh` le pasa a BibTeX la carpeta del informe en formato de Windows.	Desde Git Bash, MiKTeX no entendía la ruta y usaba otro `references.bib` de sus paquetes: ninguna cita se resolvía.
19/10/2026	Informe con `lmodern`, `microtype` y cortes después de los guiones bajos.	Fuentes vectoriales y ningún nombre de función fuera del margen.

8. Verificación y cómo reproducir

- `pytest`: 23 pruebas superadas, 3 pendientes (1.5, 2.1 y 3.1) y 0 fallidas. Además de las cuatro obligatorias de 1.1 y 1.2, hay pruebas del generador sintético (formas, balance, margen y reproducibilidad), de las gráficas, de `extract_features` con un codificador falso (sin descargar modelos) y de la búsqueda de hiperparámetros.
- Las secciones 1.3 y 1.4 del notebook se ejecutaron de principio a fin en Windows con Python 3.14, PyTorch 2.14 y sentence-transformers 6.1.0; las figuras y tablas de este documento salen de esa ejecución.
- Este documento se compiló con MiKTeX y `latexmk` (`./build_pdf.sh review`). El informe

completo (`main.tex`) también compila: 12 páginas, sin citas sin resolver ni líneas fuera del margen. Todavía no se ha generado la documentación Doxygen, que no está instalada.

Para reproducir: instalar `requirements.txt`, abrir `notebooks/tp2_detector_spam.ipynb` y ejecutar las celdas en orden. La primera ejecución descarga el dataset y el codificador; las siguientes usan las copias guardadas en `data/`. Ejecutar la celda de configuración borra las figuras y tablas de `results/`, así que este documento se debe recompilar después de ejecutar las secciones que generan las figuras que muestra.